SUNWAY UNIVERSITY

Jeffrey Cheah Foundation
Nurturing the Seeds of Wisdom

Topic 06:
Distributed File Systems

OUR VISION: TO BE A WORLD CLASS UNIVERSI

NET3204
Distributed Systems

Chee Sun LIEW cheesunl@sunway.edu.my

Learning Outcome

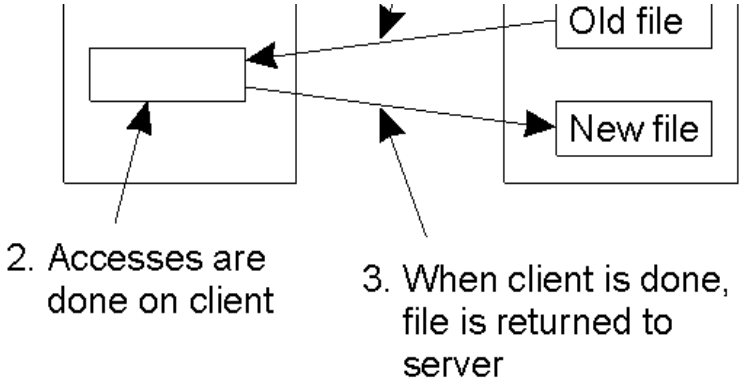
- At the end of this lecture, you will be able to:
  - Explain the concept of distributed file systems
  - List and describe the requirements and challenges of implementing distributed file systems
  - Illustrate the architecture of distributed file systems

File Service

- File is an abstraction of permanent storage, stored on non-volatile media like disks and tapes.
- Functions provided by file systems:
  - read/write data;
  - locating records based on keyword-based index.
- A file service allows programs to store and access remote files, and allows users to share files from any computer in the Internet.

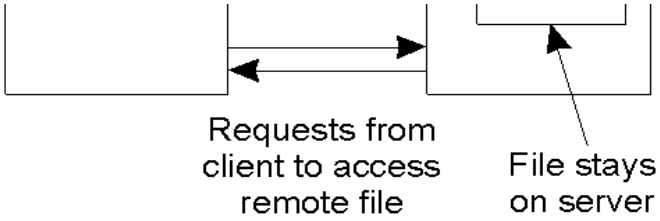
File service types

- Upload/Download model
  - Read file: copy file from server to client
  - Write file: copy file from client to server



- Advantage
  - Simple
- Problems
  - Wasteful: what if client needs small piece?
  - Problematic: what if client doesn't have enough space?
  - Consistency: what if others need to modify the same file?

File service types



- Remote access model
  - File service provides functional interface:
  - create, delete, read bytes, write bytes, etc…
- Advantages:
  - Client gets only what's needed
  - Server can manage coherent view of file system
- Problem:
  - Possible server and network congestion
  - Servers are accessed for duration of file access
  - Same data may be requested repeatedly

What is a file system?

- Persistent stored data sets
- Hierarchic name space visible to all processes
- API with the following characteristics:
  - access and update operations on persistently stored data sets
  - Sequential access model (with additional random facilities)
- Sharing of data between users, with access control
- Concurrent access:
  - certainly for read-only access
  - what about updates?
- Other features:
  - mountable file stores
  - more? ...

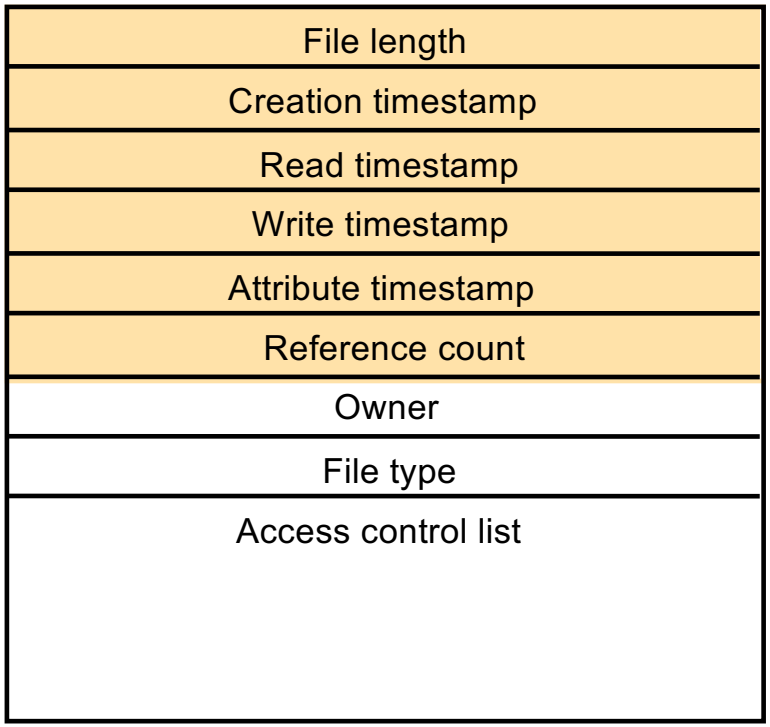
File systems modules

- Directory module: maps file names to file identifiers.
- File module: maps file identifiers to particular files.
- Access control method: checks permission for a requested operation.
- File access: read or write file data or attributes.
- Block module: accesses and allocates disk blocks.
- Device module: disk I/O buffering.

Characteristics of File Systems

- File systems are responsible for the organisation, storage retrieval, naming, sharing and protection of files.
- Files contain data and attributes.
- Metadata refers to the additional info stored by a file system that is used for file management.
  - Includes file attributes, directories and other persistent info.

File attribute record structure



- Naming of files is supported by the use of directories:
  - A directory is a special file type.
  - Provides a mapping from text names to internal file identifiers.
  - May include other directories, leading to a hierarchic file naming scheme, e.g. /lecture/wres3101/note1.pdf.

UNIX file system operations

filedes = open(name, mode) *filedes = creat(name, mode)*	Opens an existing file with the given *name*. Creates a new file with the given *name*. Both operations deliver a file descriptor referencing the open file. The *mode* is *read*, *write* or both.
status = close(filedes)	Closes the open file *filedes*.
count = read(filedes, buffer, n)	Transfers *n* bytes from the file referenced by *filedes* to *buffer*.
count = write(filedes, buffer, n)	Transfers *n* bytes to the file referenced by *filedes* from buffer. Both operations deliver the number of bytes actually transferred and advance the read-write pointer.
pos = lseek(filedes, offset, whence)	Moves the read-write pointer to offset (relative or absolute, depending on *whence*).
status = unlink(name)	Removes the file *name* from the directory structure. If the file has no other names, it is deleted.
status = link(name1, name2)	Adds a new name (*name2*) for a file (*name1*).
status = stat(name, buffer)	Gets the file attributes for file *name* into *buffer*.

Distributed File Service Requirements

- Distributed file service provides support for sharing of info.
  - Enables users to access remote files without downloading them to a local disk.
  - Provides access to files from diskless nodes.
  - Provides binder, authentication and print services.
- A file service is a critical service because it is heavily used.

Distributed File Service Requirements

- Transparency requirements.
  - Access transparency: Clients should be unaware of the distribution of files. The same command or operation can be used to access local and remote files.
  - Location transparency: Clients should see a uniform file name space. Files may be relocated without changing their pathnames.
  - Mobility transparency: Whenever files are moved, client programs need not be recompiled and system administration tables need not be changed.

Distributed File Service Requirements

- Performance transparency: Clients should continue to perform satisfactorily for a service load of a specified range.
- Scaling transparency: The service can scale to cater for increasing number of users and expansion of network size.

- Concurrent file update
  - Clients accessing a file concurrently should not be aware of the fact. Operations executed by one client should not interfere with other clients' operations. Techniques for implementing concurrency is expensive; usually implemented as file or record locking.

Distributed File Service Requirements

- File Replication
  - Copies of a file may be stored at different locations. Load of providing a service is distributed between a few servers, thus, enhancing scalability and fault tolerance.
- Fault tolerance
  - Deals with correct operation of servers after the failure of a client, and correct operation of client programs due to lost messages and temporary interruptions of service.

Distributed File Service Requirements

- Hardware and OS heterogeneity
  - The service interface can be provided on different OS and computers.
- Consistency
  - With one-copy update semantics, the file content seen by users accessing/updating a file is what they would see if only one copy of the file exists.
- Security
  - Users must be authenticated before access is granted.
  - Request/reply messages are encrypted to protect the content.

Distributed File Service Requirements

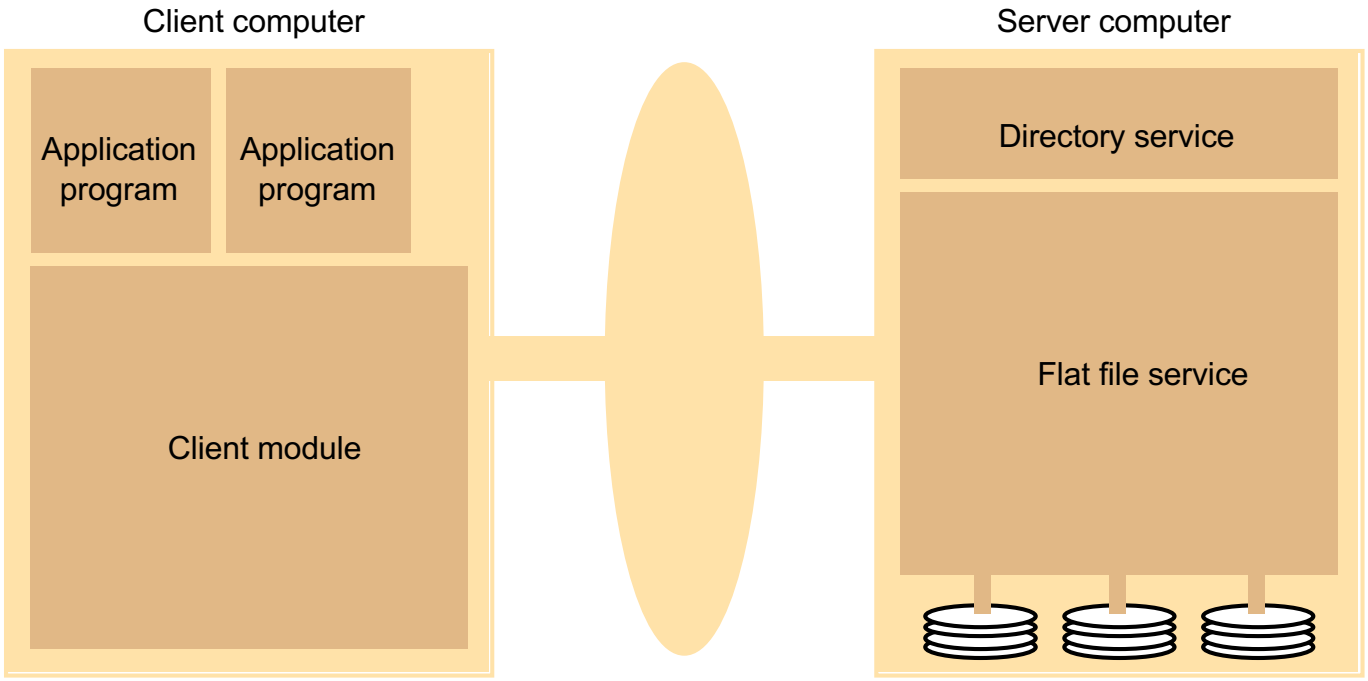
- Efficiency
  - A distributed file service should offer the same facilities offered by conventional file systems, and it should be of comparable performance and is as powerful.

- Current file services lack :
  - Support for fine-grained distribution of data. There is an increasing requirement for sharing small units of data; objects may need to be cached. File services designed for a centralised system does not cater for this requirement.
  - Tolerance to network partitioning and detached operations. Network partitioning may be caused by a link or node failure. Many replication algorithms require a majority of replicas to respond to an update request. If a partition occurs, a majority may not be available.

File Service Architecture

- There are 3 components:
  - flat file service;
  - directory service;
  - client module.

File service architecture



Flat file service

- Implements operations on the file content.
- A unique file identifier (UFID) is used to refer to files in all requests.
- Division of responsibility between the file service and the directory service is based upon the use of UFIDs.

Flat file service operations

Read(FileId, i, n) -> Data — throws *BadPosition*	If *1 ≤ i ≤ Length(File)*: Reads a sequence of up to *n* items from a file starting at item *i* and returns it in *Data*.
Write(FileId, i, Data) — throws *BadPosition*	If *1 ≤ i ≤ Length(File)+1*: Writes a sequence of *Data* to a file, starting at item *i*, extending the file if necessary.
Create() -> FileId	Creates a new file of length 0 and delivers a UFID for it.
Delete(FileId)	Removes the file from the file store.
GetAttributes(FileId) -> Attr	Returns the file attributes for the file.
SetAttributes(FileId, Attr)	Sets the file attributes (only those attributes that are not shaded in).

Directory service

- Maps the text name of a file to its UFID.
- Services provided:
  - Generate and update directories;
  - Add new file names to directories;
  - Entertain requests for UFIDs.
- Clients may query the directory service for the UFID of a file by quoting its text name.
- Is a client of the flat file service - its directory files are stored as files by the flat file service.

Directory service operations

Lookup(Dir, Name) -> FileId — throws *NotFound*	Locates the text name in the directory and returns the relevant UFID. If *Name* is not in the directory, throws an exception.
AddName(Dir, Name, File) — throws *NameDuplicate*	If *Name* is not in the directory, adds (*Name*, *File*) to the directory and updates the file's attribute record. If *Name* is already in the directory: throws an exception.
UnName(Dir, Name) — throws *NotFound*	If *Name* is in the directory: the entry containing *Name* is removed from the directory. If *Name* is not in the directory: throws an exception.
GetNames(Dir, Pattern) -> NameSeq	Returns all the text names in the directory that match the regular expression *Pattern*.

Client module

- Runs on each client computer.
- Holds info about the location of the flat file server and directory server processes.
- Integrates and extends the flat file service and the directory service and present them to user-level programs under one interface.
- Implements caching of recently used file blocks to improve performance.

Access Control

- IN Unix, a user's access right is checked against the access mode.
- Two approaches to ensure only authorised users are granted access:
  - An access check is performed when a file name is converted to a UFID and the results are encoded in form of a capability which is returned to the client for use in future requests.
  - A user's identity is submitted with every request and access checks are performed for each requested operation.

Hierarchic file systems

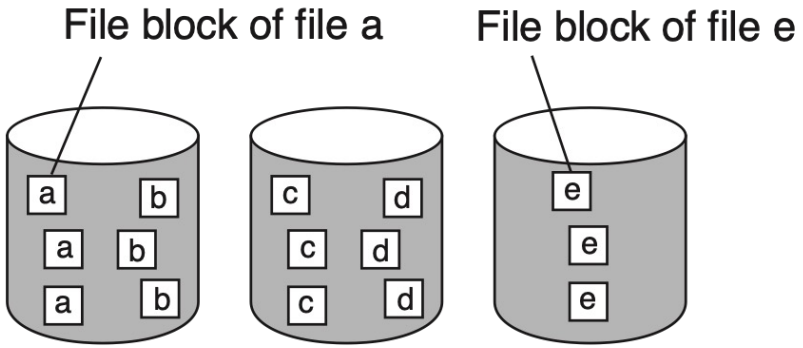
- Directories are arranged in a tree structure.
- Each directory holds the names of files and other directories that are accessible from it.
- A tree-structured network of directories is constructed with files at the leaves and directories at other nodes of the tree.
- The root of the tree is a directory with a well-known UFID.

Case Studies

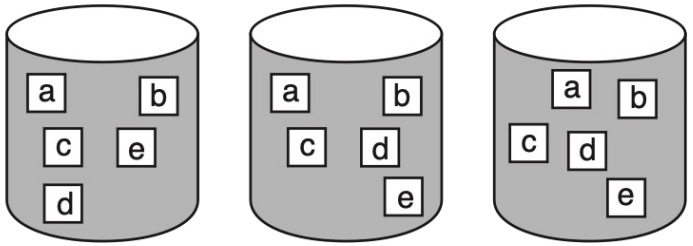
GFS, Sun NFS, AFS

Cluster-Based File Systems

- When dealing with very large data collections, following a simple client-server approach is not going to work
  - for speeding up file accesses, apply striping techniques by which files can be fetched in parallel



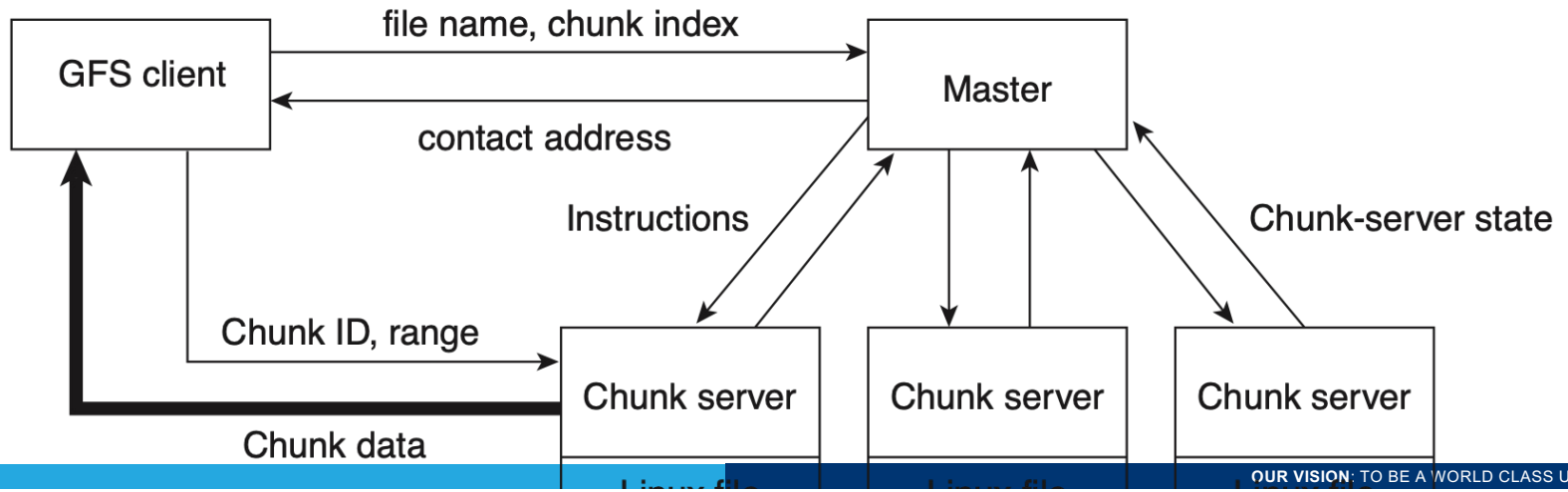
whole-file distribution



file-striped system

Example: Google File System

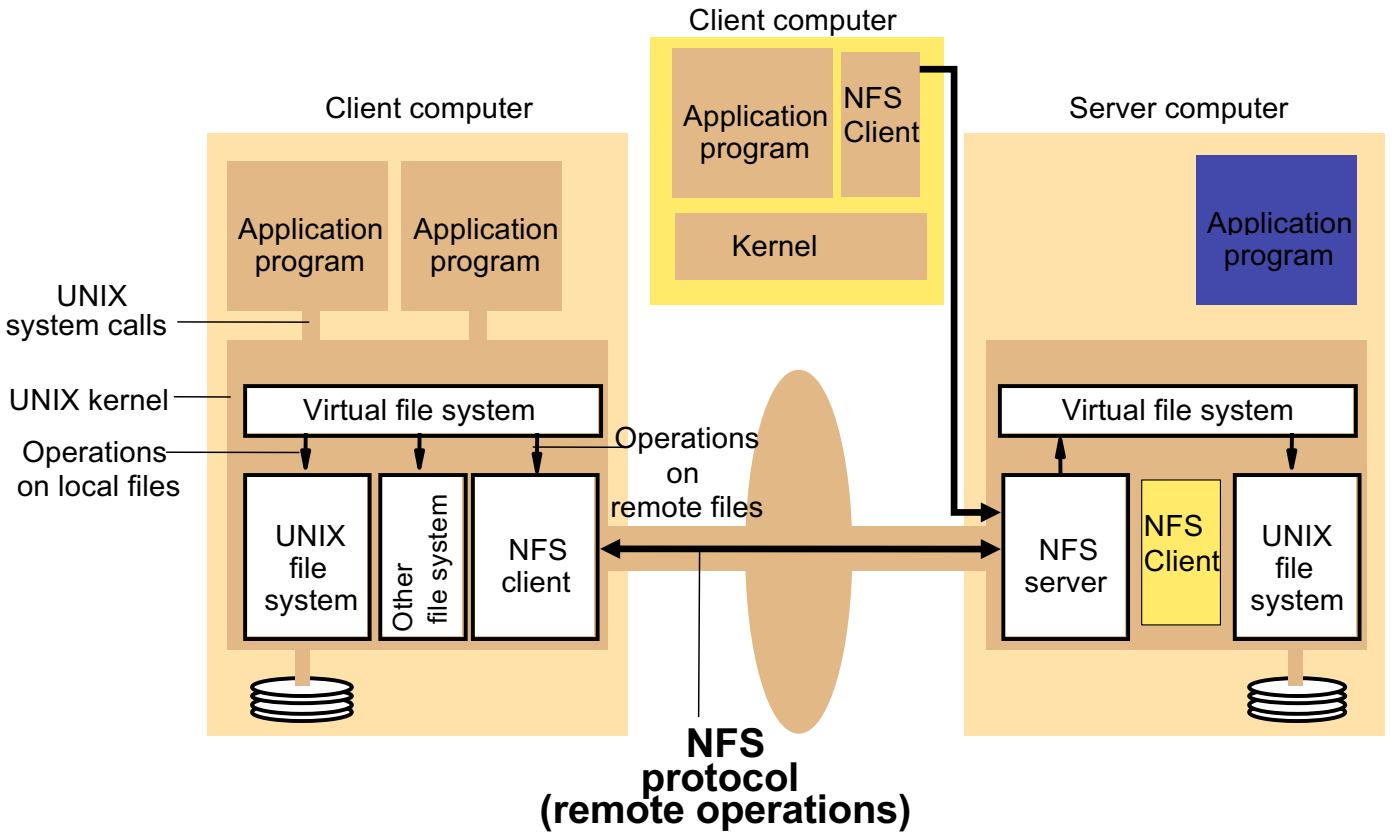
- Divide files in large 64 MB chunks, and distribute/replicate chunks across many servers:
  - The master maintains only a (file name, chunk server) table in main memory >>> minimal I/O
  - Files are replicated using a primary-backup scheme; the master is kept out of the loop



Sun NFS

- NFS protocols are independent of OS.
- Provides transparent access to remote files.
- Usually, each computer has an NFS client and server modules are installed in the kernel.
- Client-server relationship is symmetrical: each computer can act as a client and a server, and provides remote access for other machines.
  - Common to configure a few machines as dedicated servers.

NFS architecture



OUR VISION: TO BE A WORLD CLASS UNIVERSITY

An incomplete list of file system operations supported by NFS

Operation	v3	v4	Description
Create	Yes	No	Create a regular file
Create	No	Yes	Create a nonregular file
Link	Yes	Yes	Create a hard link to a file
Symlink	Yes	No	Create a symbolic link to a file
Mkdir	Yes	No	Create a subdirectory in a given directory
Mknod	Yes	No	Create a special file
Rename	Yes	Yes	Change the name of a file
Rmdir	Yes	No	Remove an empty subdirectory from a directory
Open	No	Yes	Open a file
Close	No	Yes	Close a file
Lookup	Yes	Yes	Look up a file by means of a file name
Readdir	Yes	Yes	Read the entries in a directory
Readlink	Yes	Yes	Read the path name stored in a symbolic link
Getattr	Yes	Yes	Read the attribute values for a file
Setattr	Yes	Yes	Set one or more attribute values for a file
Read	Yes	Yes	Read the data contained in a file
Write	Yes	Yes	Write data to a file

- NFS provides a high level of support for hardware and OS heterogeneity.
  - Implemented on most version of Unix.
  - Also available for IBM PC and Apple Macintosh.
- NFS success in achieving transparency:
  - Access transparency: Local and remote files are accessed in an identical manner. Existing programs need not be modified in order to operate correctly with remote files.

Implementation of NFS

- The server module on each computer acts as a NFS server.
- Requests for remote files are translated by the client module to NFS protocol operations and passed to the server module.
- Clients and servers communicate using RPC.
  - Clients use a port mapper service to obtain binding to a server.

- Encryption is provided as an optional security feature.
- Any process can send requests to a server; access is granted if requests are valid and contain the correct authentication info.
- NFS clients and servers are OS-independent and can be implemented on non-Unix environments.

- When a client uses a multipart pathname, the client module must parse the name and look up each part separately, stepping through the name tree.
  - The pathname may traverse several file systems residing on different servers or in the client itself.
  - Also helps achieve OS heterogeneity because parsing of names may use a name syntax specific to the client OS.

Virtual File System (VFS)

- A VFS module was added to the Unix kernel to:
  - Distinguish between local and remote files;
  - Translate between Unix-independent file identifier by NFS and internal file identifier used in Unix and other file systems.
- File identifiers are called file handles in NFS.
  - Is opaque to clients.
  - Contains information the server needs to distinguish individual files.

- A file handle consists of:
  - file system id: a unique number allocated to a file system when it is created;
  - file i-node number: a number used by the server to identify and locate the file within a file system;
  - i-node generation number: conventional Unix reuses an i-node number after a file is removed. A generation number is stored in each file and incremented each time the i-node number is reused.

Client integration

- NFS client module provides an interface to application programs.
- Emulates the semantics of the standard Unix file system and is integrated with the Unix kernel.
- The client module cooperates with the virtual file system in each client machine.
  - Transfers blocks of file to and from the server.
  - Caches blocks in memory whenever possible ⇒ gives rise to cache consistency problem.

- Server integration
- The server module is integrated with the Unix kernel to achieve good performance.
- Access control and authentication
- Since the server is stateless, it does not keep files open for clients.
  - A client's identity and access permission is checked each time a request is sent.
- Clients are required to send user authentication info in each request.

- Security loophole:
  - A client can impersonate another client by including the authentication info in its request.
  - This problem is overcome by using DES to encrypt user's authentication info.

Mount service

- Is a service to mount remote file stores for clients.
- An access list maintains a list of hosts permitted to mount a file system.
- When requesting a file to be mounted, the client specifies:
  - the remote host name,
  - pathname of a directory at the remote host, and
  - a local name to which it is to be mounted.

Mount service

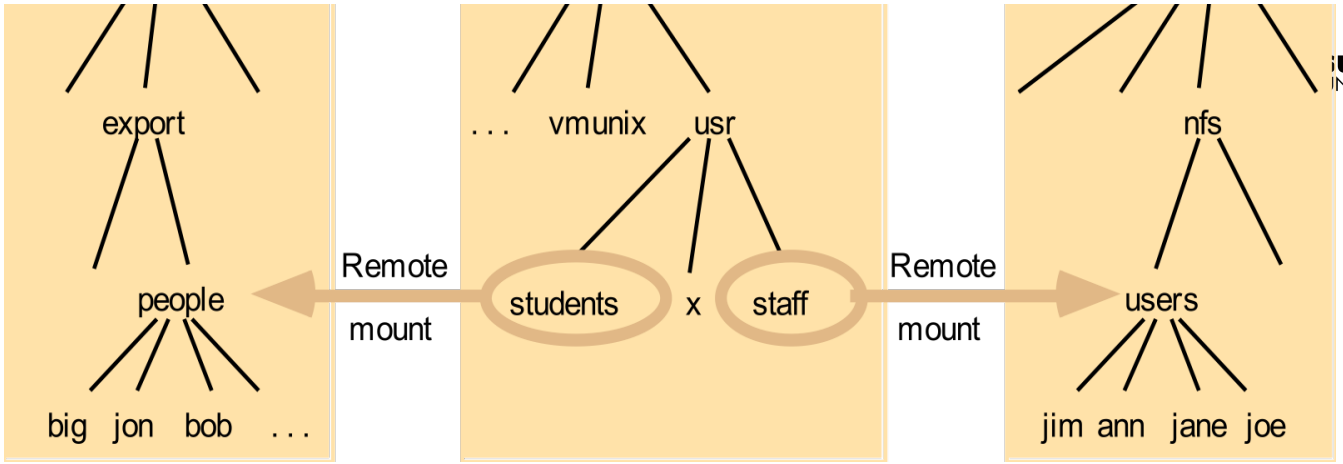
- Mount operation:

```
mount(remotehost, remotedirectory, localdirectory)
```

- Server maintains a table of clients who have mounted filesystems at that server
- Each client maintains a table of mounted file systems holding:

  `< IP address, port number, file handle>`

- Hard versus soft mounts



- Node people is mounted over node students. Client can access files at Server 1 by using pathname such as /usr/students/jon.
- Node users at Server 2 is mounted over node staff in client's local file store. Client can access files at Server 2 by using pathname such as usr/staff/ann.

Path name translation

- Multipart pathnames are translated into i-node reference in stages.
- The process is handled at the client.
- If a part of a name refers to a remote directory, it is translated to a file handle using a separate lookup request at the remote server.
- At each step, the remote mount table in the client is checked to see whether another remote mounted file store has to be accessed.
- The result from each step is cached.

Automounter

- Is a service which dynamically mounts a file system whenever an empty mount point is referenced by a client.
- Runs on each client computer.
- Maintains a table of mount points/pathnames which refer to one or more servers.
- When a client module attempts to resolve a pathname containing a mount point:
  - It passes a lookup request to the automounter;

- The automounter locates the file system in its table and sends a request to each server listed;
- The file system on the first server to respond is mounted using the normal mount service;
- The mounted file system is linked to the mount point using a symbolic link.

Server caching

- In Unix, file pages, directories and file attributes that have been read from a disk are kept in a buffer cache until the space is required by other pages.
  - If later a request is issued for a page already in the cache, access can be made to the cache.
- Read-ahead: Anticipates read accesses and fetches the following pages based on most recently been read pages.

- Delayed write: An updated page is only written to disk when the buffer space is required for another page.
  - Sync operation flushes altered pages to disk every 30 seconds.
- These caching techniques work in a UNIX environment because all read and write requests are passed to a single cache implemented in the Unix kernel space.
  - Cache is always updated;
  - File accesses cannot bypass the cache.

- NFS server uses a similar caching mechanism.
  - Write-through: When a page is updated, it is written to disk immediately because if the server fails, the loss of data is undetected by clients.
- Client caching
- The client module caches the result of read, write, getattr, lookup and readdir to reduce the number of requests sent to a server.

- Caching at clients gives rise to cache consistency problem:
  - Write operation at one client causes other clients' caches to become stale and caches are not updated immediately.
- Timestamp-based technique is used to minimise inconsistency.
  - TC is the time cache was last validated and TM is time cache was last modified.
  - A cache entry is valid at time T if:
    - (T-Tc < t) OR (TMclient = TMserver) where t is the time interval. If the first check is true the second one need not be evaluated.

- A validity check is performed whenever a cache entry is used or client receive results of operation on a file (piggybacked).

- Caches are assumed valid for a fixed time specified where no remote access to server involved.
  - 3s-30s for files; 30s-60s for directories.
  - A reference to the file after the time elapses causes another validation check.

- When a cached page is updated, it is marked as dirty and is scheduled to be flushed to the server asynchronously.
  - Dirty caches are flushed when a file is closed, or a sync occurs at the client.
- Clients cannot determine if a file is shared.
  - A validation procedure must be used for all files accesses, but does not guarantee consistency because recent updates are not always visible to clients.

- 2 sources of time lag:
  - The delay between the write operation and the update data being flushed to the server,
  - 3s window for cache validation.

- Bio-daemon processes are used to implement read-ahead and delayed-write asynchronously.
  - A bio-daemon is notified after each read request, and it requests the transfer of the following file block from the server to the client cache.
  - Whenever a block is updated, it sends the block to the server. Directory blocks are sent whenever a modification occurs.

Performance

- Use of NFS does not impose significant performance penalty.
- 2 problems:
  - Frequent use of getattr to fetch timestamps for cache validation.
  - Poor performance of write operations because write-through is used at the server, but since write operations are relatively infrequent, the poor performance is tolerable.
- Lookup operation account for almost 50% of server calls due to step-by-step pathname translation.

Achievement of transparencies

- Access: Excellent; the API is the UNIX system call interface for both local and remote files.
- Location: Not guaranteed but normally achieved; naming of filesystems is controlled by client mount operations, but transparency can be ensured by an appropriate system configuration.
- Concurrency: Limited but adequate for most purposes; when read-write files are shared concurrently between clients, consistency is not perfect.

Achievement of transparencies (cont.):

- Replication: Limited to read-only file systems; for writable files, the SUN Network Information Service (NIS) runs over NFS and is used to replicate essential system files.
- Failure: Limited but effective; service is suspended if a server fails. Recovery from failures is aided by the simple stateless design.
- Mobility: Hardly achieved; relocation of files is not possible, relocation of filesystems is possible, but requires updates to client configurations.

Achievement of transparencies (cont.):

- Performance: Good; multiprocessor servers achieve very high performance, but for a single filesystem it's not possible to go beyond the throughput of a multiprocessor server.
- Scaling: Good; filesystems (file groups) may be subdivided and allocated to separate servers. Ultimately, the performance limit is determined by the load on the server holding the most heavily-used filesystem (file group).

Achievement of transparencies

- Access: Excellent; the API is the UNIX system call interface for both local and remote files.
- Location: Not guaranteed but normally achieved; naming of filesystems is controlled by client mount operations, but transparency can be ensured by an appropriate system configuration.
- Concurrency: Limited but adequate for most purposes; when read-write files are shared concurrently between clients, consistency is not perfect.

Achievement of transparencies (cont.):

- Replication: Limited to read-only file systems; for writable files, the SUN Network Information Service (NIS) runs over NFS and is used to replicate essential system files.
- Failure: Limited but effective; service is suspended if a server fails. Recovery from failures is aided by the simple stateless design.
- Mobility: Hardly achieved; relocation of files is not possible, relocation of filesystems is possible, but requires updates to client configurations.

Achievement of transparencies (cont.):

- Performance: Good; multiprocessor servers achieve very high performance, but for a single filesystem it's not possible to go beyond the throughput of a multiprocessor server.
- Scaling: Good; filesystems (file groups) may be subdivided and allocated to separate servers. Ultimately, the performance limit is determined by the load on the server holding the most heavily-used filesystem (file group).

Summary

- What did I learn in this module?
  - The concept of distributed file systems
  - The design requirements and challenges of distributed file systems
  - The architecture of distributed file systems
  - An example distributed file systems - Sun NFS

Acknowledgement

The figures and contents used in this lecture are adopted and adapted from:

- van Steen, M. and Tanenbaum A.S. (2023). Distributed Systems, 4th Edition. distributed-systems.net.
- Colouries, G., Dollimore, J., Kindberg, T and Blair, G. (2021). Distributed Systems: Concepts and Design, 5th Edition. Pearson.
- Web references.

THANK YOU

Chee Sun LIEW
cheesunl@sunway.edu.my

